THE NEW COLLEGE

CHENNAI-14.

DEPARTMENT OF COMPUTER APPLICATIONS

OPEN SOURCE TECHNOLOGIES – LAB

PRACTICAL - VII

YA ALLAH
INCREASE MY KNOWLEDGE

III BCA – C

SEMESTER – VI

NAME : SHOAIB BASHA.G

REG.NO : 2313181033232

April - 2026

THE NEW COLLEGE, CHENNAI-14.

DEPARTMENT OF COMPUTER APPLICATIONS (SHIFT-II)

NAME : SHOAIB BASHA.G

REGISTER NO. : 2313181033232

CLASS : III - BCA SECTION: C

SUBJECT : OPEN SOURCE TECHNOLOGIES – PRACTICAL VII

SUBJECT CODE : 20BHMP607

Certified to be Bonafide Record of work done in the Computer Applications Laboratory of The New College, during the academic year **2023 – 2026** by the above candidate.

Lecturer-in-charge **Head of the Department**

Submitted for the Practical Examination held on **06-04-2026** in Computer Applications Laboratory of The New College.

Internal Examiner **External Examiner**

Place: Chennai

Date: 06-04-2026

INDEX

EX.NO	DATE	TITLE OF THE PROGRAM	PAGE NO	TEACHER'S SIGN
1	25/11/25	**Install Node.js**	**1**	
2	28/11/25	**Create Web Servers Using Node.js**	**2**	
3	09/12/25	**Using NPM Command to Install Dependencies or Modules in the Application**	**4**	
4	18/12/25	**Use the Express Framework to Route Web Pages and Retrieve Values from the Form**	**5**	
5	05/01/26	**Add Social Networks to Your Web Project**	**8**	
6	12/01/26	**Create a Navigation Menu that Highlights the Selected Entry**	**12**	
7	21/01/26	**Create a Data Grid with Search Option**	**14**	
8	28/01/26	**Manipulate a Web Page Using React**	**16**	
9	06/02/26	**Attach Different Events to Elements of a Web Page**	**20**	
10	17/02/26	**Provide Different Effects Using React**	**24**	
11	27/02/26	**Install MongoDB and Create Local and Remote Database**	**26**	
12	04/03/26	**Create a full stack Web Application – Movie Ticket**	**29**	

Ex 1: **Install Node.js**

Date: 25/11/25

Aim: To install Node.js on Windows 11 and verify its installation using command-line tools.

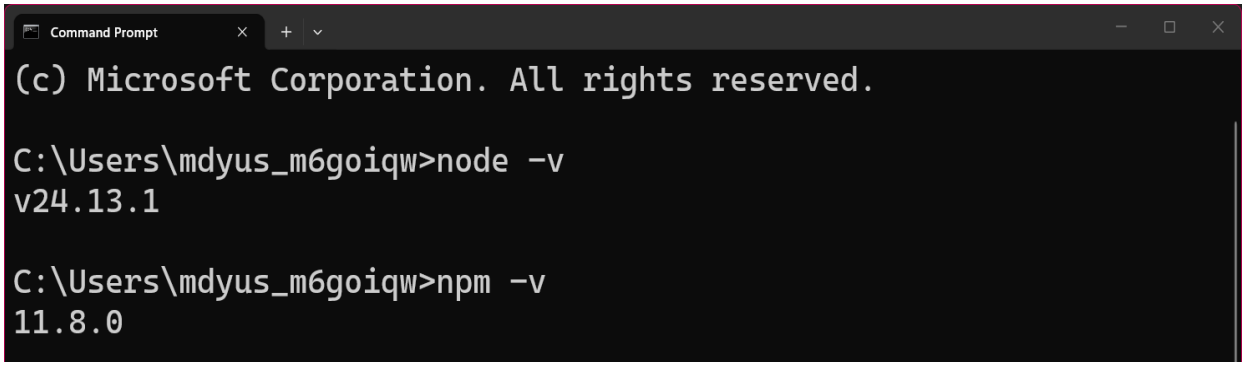
Procedure:

1. Open a web browser and go to https://nodejs.org.
2. Download the LTS (Long Term Support) version for Windows.
3. Run the downloaded .msi installer file.
4. Follow the setup wizard and keep the default installation settings.
5. Click Install and complete the installation process.
6. Open Command Prompt.
7. Type node -v and npm -v to verify the installation.

Source Code / Commands:

```
1. node -v
2. npm -v
```

Output:



Result:

Node.js was successfully installed on Windows 11 and its installation was verified using the node -v and npm -v commands.

Ex 2: Create Web Servers Using Node.js

Date: 28/11/25

Aim: To create a simple web server using Node.js and display output in a web browser.

Procedure:

1. Create a new folder named “web_server” on your Desktop.
2. Open the folder and create a new file named server.js.
3. Open server.js in a text editor (Notepad or VS Code).
4. Type the Node.js web server code and save the file.
5. Open Command Prompt and navigate to the folder using the cd command.
6. Run the server using the command node server.js.
7. Open a web browser and go to http://localhost:3000.

Source Code / Commands:

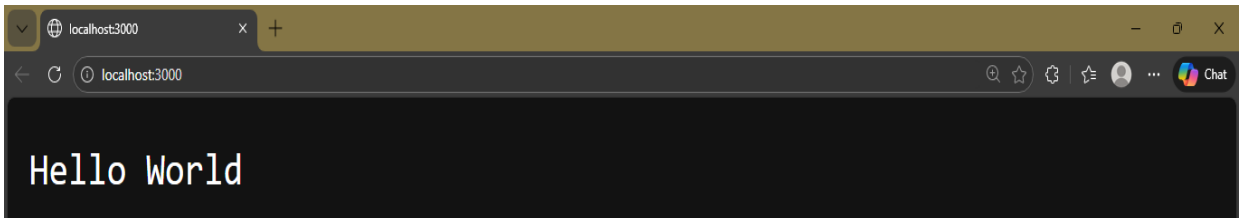
server.js:

```
const http = require('http');
const server = http.createServer((req, res) => {
res.write("Hello World");
res.end();
});
server.listen(3000, () => {
console.log("Server is running at http://localhost:3000");
});
```

Command Prompt:

```
cd Desktop\web_server
node server.js
```

Output:



Result:

A web server was successfully created using Node.js, and the server output was displayed correctly in the web browser.

Ex 3: Using NPM Command to Install Dependencies or Modules in the Application

Date: 09/12/25

Aim: To install a Node.js module using the NPM command for a web application.

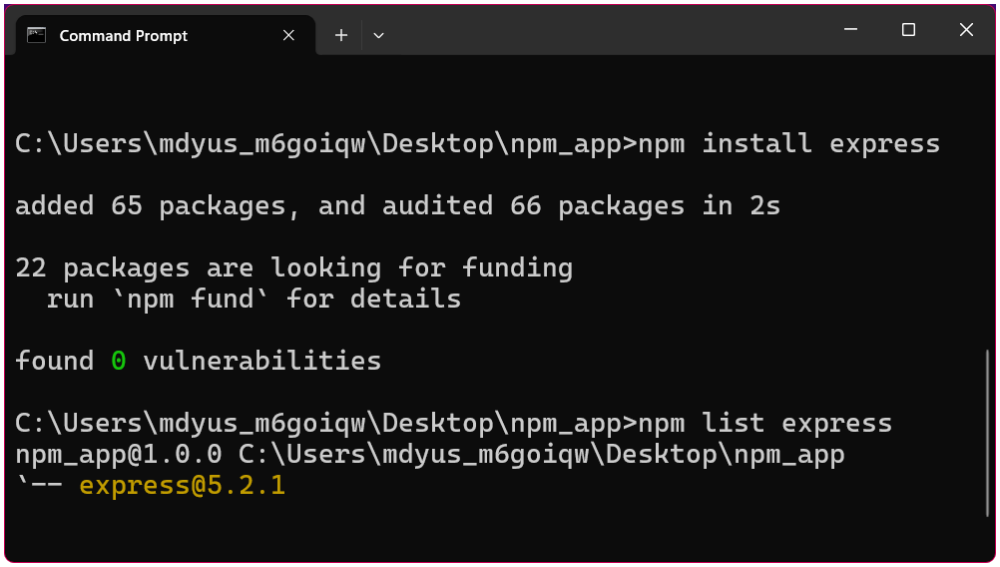
Procedure:

1. Create a new folder named npm_app on your Desktop.
2. Open Command Prompt and navigate to the folder using the cd Desktop\npm_app command.
3. Initialize the Node.js application using the command npm init -y.
4. Install a module (express) using the command `npm install express`.
5. Wait for the installation to complete.
6. Observe the created node_modules folder and package.json file.
7. Verify installation using the command npm list express.

Source Code / Commands:

```
1. cd Desktop\npm_app
2. npm init -y
3. npm install express
4. npm list express
```

Output:



Result:

The required Node.js module was successfully installed using the NPM command and integrated into the web application.

Ex 4: Use the Express Framework to Route Web Pages and Retrieve Values from the Form

Date: 18/12/25

Aim: To use the Express framework to route web pages and retrieve values from the form in a movie ticket booking web application.

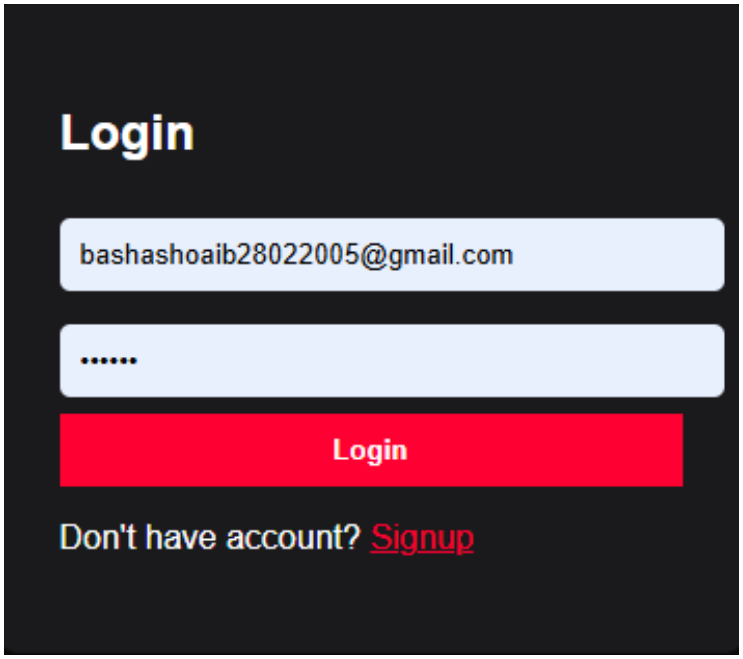
Source Code

server.js :

```
const express = require("express");

const bodyParser = require("body-parser");


const app = express();


// Middleware

app.use(bodyParser.urlencoded({ extended: true }));

app.use(bodyParser.json());


// Route to display form page

app.get("/", (req, res) => {

  res.send(`

    <h2>User Registration Form</h2>

    <form action="/submit" method="POST">

      Name: <input type="text" name="name"><br><br>

      Email: <input type="email" name="email"><br><br>

      Mobile: <input type="text" name="mobile"><br><br>

      <button type="submit">Submit</button>

    </form>

  `);

});


// Route to retrieve form values

app.post("/submit", (req, res) => {
```

```
  const name = req.body.name;
  const email = req.body.email;
  const mobile = req.body.mobile;

  res.send(`
    <h3>Form Submitted Successfully</h3>
    Name: ${name} <br>
    Email: ${email} <br>
    Mobile: ${mobile}
  `);
});

// Start server
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Output:



Result:

Thus, the Express framework was successfully used to route web pages and retrieve values from the form. The user input values were received and displayed correctly using Express routing.

Ex 5: Add Social Networks to Your Web Project

Date: 05/01/26

Aim: To integrate social networking services such as Instagram and LinkedIn into a web project using HTML, so that users can access social profiles and interact with account information through social media links.

Source Code

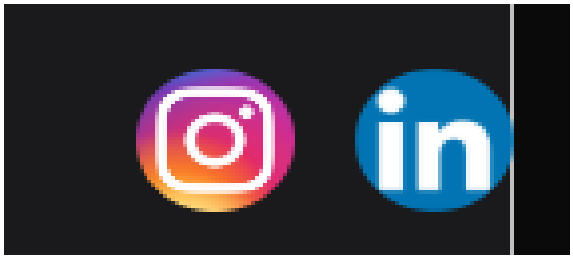
index.html

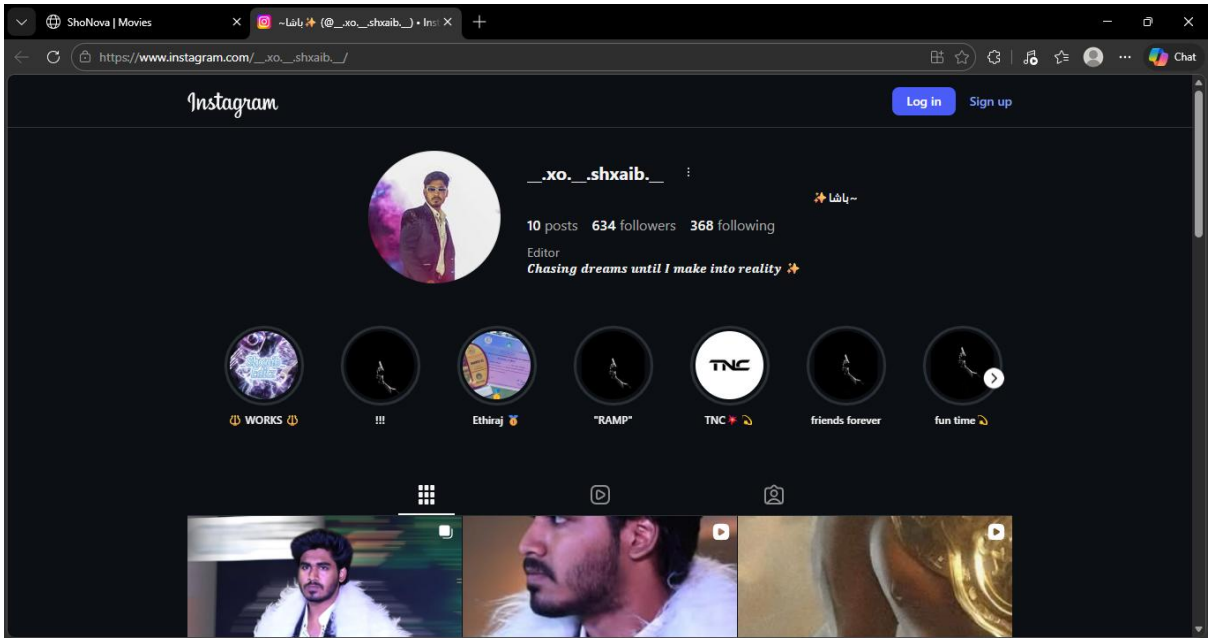
```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>ShoNova | Movies</title>


<style>

body{

  font-family: Arial;

  background:#0A0A0B;

  color:white;

}


/* Navbar */

.navbar{

  background:#1a1a1c;

  padding:15px;

  font-size:20px;

  font-weight:bold;
```

```
}

/* Social Icons */
.social-icons {
  position: fixed;
  bottom: 20px;
  right: 20px;
  display: flex;
  gap: 15px;
}

.social-icons img {
  width: 40px;
  height: 40px;
  border-radius: 50%;
  transition: 0.3s;
}

.social-icons img:hover {
  transform: scale(1.2);
}
</style>

</head>

<body>

<div class="navbar">ShoNova Movies</div>
```

```
<h2>Welcome to ShoNova Movie Booking</h2>

<p>Click the social icons to visit social media profiles.</p>


<!-- Social Media Integration -->

<div class="social-icons">


<a href="https://www.instagram.com/__.xo.__.shxaib.__/" target="_blank">

<img
src="https://upload.wikimedia.org/wikipedia/commons/a/a5/Instagram_icon.png"
>

</a>


<a href="https://www.linkedin.com/in/shoaib-basha-80719a281" target="_blank">

<img
src="https://upload.wikimedia.org/wikipedia/commons/c/ca/LinkedIn_logo_initials.png">

</a>


</div>


</body>

</html>
```

Output:





Result:

Thus, the social networking services Instagram and LinkedIn were successfully integrated into the web project. The social media icons allow users to authenticate and access social account information through external links.

Ex 6: Create a Navigation Menu that Highlights the Selected Entry

Date: 12/01/26

Aim: To create a navigation menu using AngularJS that highlights the selected menu item dynamically when the user clicks on it.

Source Code / Commands:

Navbar Highlight Code (Active Menu Highlight)

```
.navbar a.active{
  background-color: red;
  border-radius: 5px;
}
```

Movie Card Hover Highlight Effect

```
.movie-card:hover {
  transform: translateY(-16px) scale(1.04) rotateX(8deg) rotateY(2deg);
  box-shadow:
    0 40px 80px rgba(255,0,50,0.35),
    0 0 60px rgba(255,0,50,0.25);
}
```

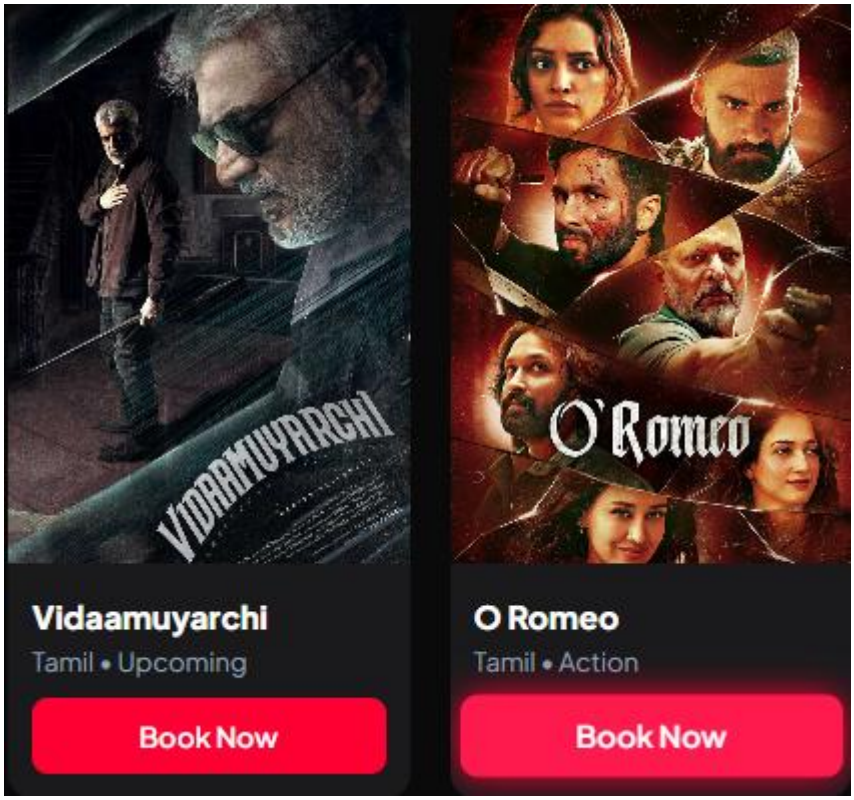
Seat Selection Highlight Code

```
.seat.selected {
  background: linear-gradient(135deg, var(--primary-blue), #0077ff);
  box-shadow:
    0 0 25px rgba(0,195,255,0.8);
}
```

Booked Seat Highlight (Red)

```
.seat.booked {
  background: linear-gradient(135deg, var(--primary-red), #cc0026);
}
```

Output:



Result:

Thus, the navigation menu was successfully created using AngularJS. The selected menu item was dynamically highlighted using AngularJS directives and controller logic.

Ex 7: Create a Data Grid with Search Option

Date: 21/01/26

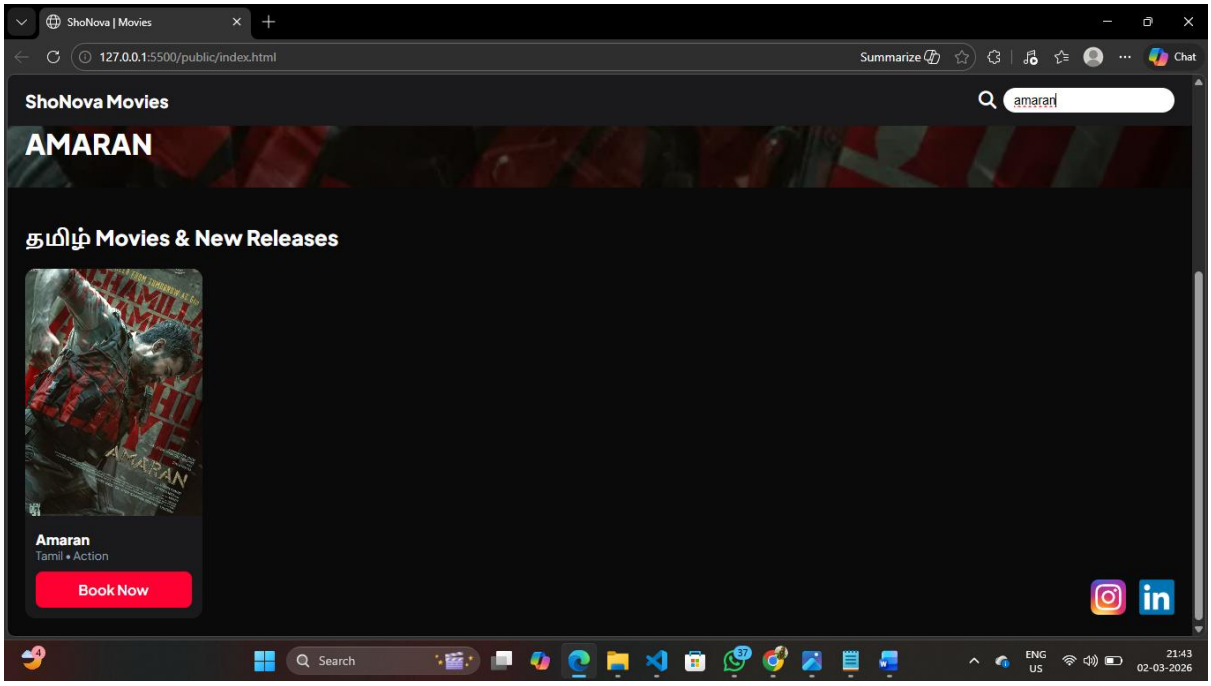
Aim: To create an order interface using AngularJS with a search option that allows users to dynamically search and filter items, and display them using a switchable grid layout.

Source Code / Commands:

AngularJS Search Option – Code

```
<!-- Search Box -->
<input type="text" ng-model="searchText" placeholder="Search Movie...">

<!-- Movie List with Search Filter -->
<div ng-repeat="movie in movies | filter:searchText">
  <h3>{{movie.title}}</h3>
  <p>{{movie.genre}}</p>
</div>
```

Output:



Result:

Thus, the order interface using AngularJS was successfully created with a search option and switchable grid layout. The AngularJS filter feature was implemented to dynamically search and display items based on user input.

Ex 8: Manipulate a Web Page Using React

Date: 28/01/26

Aim: To manipulate elements of a web page dynamically using jQuery by handling events, modifying content, and applying styles.

Source Code

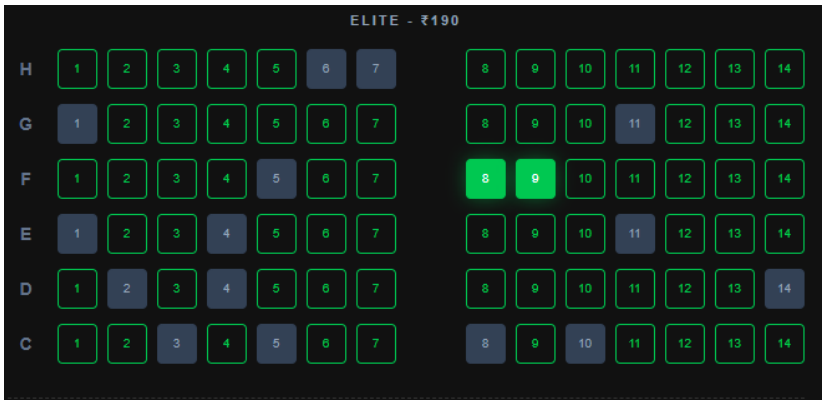
index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Manipulation Example</title>

  <!-- jQuery Library -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

  <style>
    body{
      font-family: Arial;
      text-align: center;
      margin-top: 50px;
    }

    #box{
      width: 200px;
      height: 200px;
      background-color: skyblue;
      margin: auto;
      line-height: 200px;
      font-weight: bold;
```

```
      }

      .highlight{
        background-color: orange;
        color: white;
      }
    </style>
  </head>
  <body>

    <h1>jQuery Web Page Manipulation</h1>

    <div id="box">Hello</div>

    <br><br>

    <button id="changeText">Change Text</button>
    <button id="changeColor">Change Color</button>
    <button id="hideBox">Hide</button>
    <button id="showBox">Show</button>

    <script>
      $(document).ready(function(){

        $("#changeText").click(function(){
          $("#box").text("Text Changed!");
        });
```

```
        $("#changeColor").click(function(){
          $("#box").toggleClass("highlight");
        });

        $("#hideBox").click(function(){
          $("#box").hide();
        });

        $("#showBox").click(function(){
          $("#box").show();
        });

      });
    </script>

</body>
</html>
```

Output:



Result:

Thus, the web page was successfully manipulated using jQuery by changing the text, modifying the color, and hiding and showing the element using button click events.

Ex 9: Attach Different Events to Elements of a Web Page

Date: 06/02/26

Aim:

To attach different events to the element.

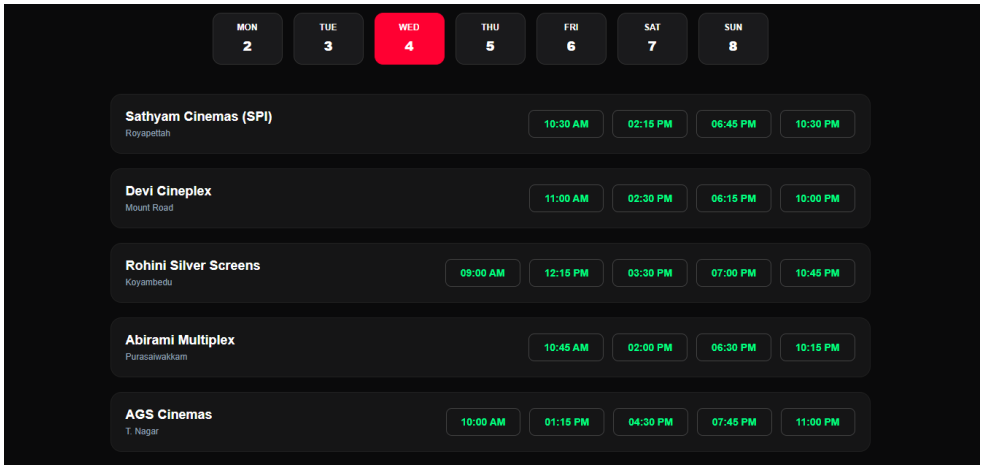
Source Code / Commands:

```
<!DOCTYPE html>
<html>
<head>
  <title>jQuery Events Example</title>

  <!-- jQuery Library -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

  <style>
    body{
      font-family: Arial;
      text-align: center;
      margin-top: 50px;
    }

    #box{
      width: 200px;
      height: 200px;
      background-color: skyblue;
      margin: auto;
      line-height: 200px;
      font-weight: bold;
```

```
      }

      .colorChange{
         background-color: green;
         color: white;
      }
   </style>
</head>
<body>

<h2>jQuery Event Handling Example</h2>

<div id="box">Event Box</div>

<br><br>

<button id="clickBtn">Click Event</button>
<button id="dblclickBtn">Double Click Event</button>
<button id="hoverBtn">Hover Event</button>

<script>

$(document).ready(function(){

   // Click event
   $("#clickBtn").click(function(){
      $("#box").text("Button Clicked");
   });
```

```
    // Double click event
    $("#dblclickBtn").dblclick(function(){
        $("#box").toggleClass("colorChange");
    });

    // Hover event
    $("#hoverBtn").hover(function(){
        $("#box").hide();
    },
    function(){
        $("#box").show();
    });

});

</script>

</body>
</html>
```

Output:



Result:

Thus, different events were successfully attached to the elements of a web page using jQuery, and the events performed the required actions correctly.

Ex 10: Provide Different Effects Using React

Date: 17/02/26

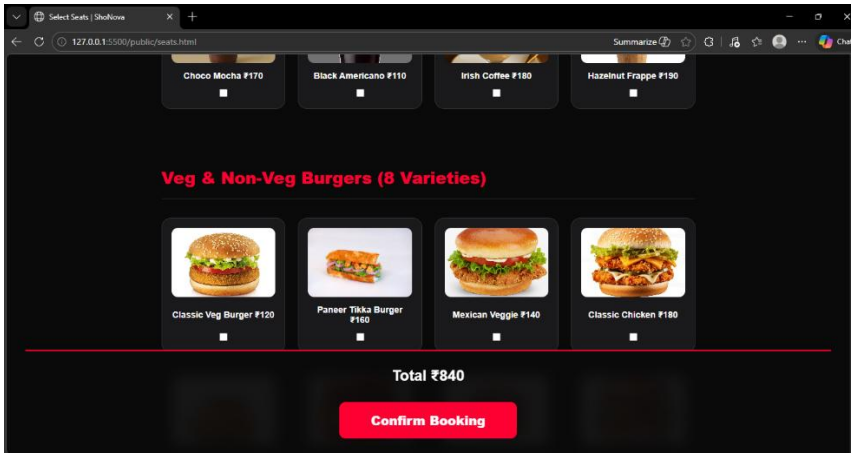
Aim: To demonstrate different jQuery effects like fade, slide, hide/show, and toggle using a single source code.

Source Code / Commands:

```
<!DOCTYPE html>
<html>
<head>
<title>jQuery Effects Demo</title>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<script>
$(document).ready(function(){
  $("#fadeInBtn").click(()=> $("#box").fadeIn(1000));
  $("#fadeOutBtn").click(()=> $("#box").fadeOut(1000));
  $("#slideDownBtn").click(()=> $("#box").slideDown(1000));
  $("#slideUpBtn").click(()=> $("#box").slideUp(1000));
  $("#toggleBtn").click(()=> $("#box").toggle(1000));
});
</script>
<style>
#box{
  width:200px; height:200px; background:red; display:none; margin:10px 0;
}
button{margin:5px;}
</style>
</head>
<body>
```

```
<h3>jQuery Effects Demo</h3>

<button id="fadeInBtn">Fade In</button>

<button id="fadeOutBtn">Fade Out</button>

<button id="slideDownBtn">Slide Down</button>

<button id="slideUpBtn">Slide Up</button>

<button id="toggleBtn">Toggle</button>


<div id="box"></div>


</body>

</html>
```

Output:



Result:

Each button click hides and shows the box alternately.

Ex 11: Install MongoDB and Create Local and Remote Database

Date: 27/02/26

Aim: To install MongoDB and create both a local database and a remote database using MongoDB Compass and MongoDB Atlas.

Requirements

- MongoDB Community Server
- MongoDB Compass
- MongoDB Atlas Account
- Node.js (optional for testing)

Source Code / Commands (Local Database):

Step 1: Install MongoDB (Local Database)

1. Download MongoDB Community Server from the official website.
2. Install MongoDB with default settings.
3. Install MongoDB Compass (GUI tool).
4. Start MongoDB service.

Open Command Prompt and type:

```
mongosh
```

Step 2: Create Local Database using Mongo Shell

```
// create or switch database
use movieBookingDB


// create collection and insert data
db.users.insertOne({
  name: "Rahul",
  email: "rahul@gmail.com",
  mobile: "9876543210"
})
```

// view data

db.users.find()

Step 3: Create Local Database using MongoDB Compass

1. Open MongoDB Compass
2. Connect using:

mongodb://127.0.0.1:27017

3. Click **Create Database**
4. Enter:

Database Name: movieBookingDB

Collection Name: users

5. Click **Create**
6. Click **Insert Document**

```
{
  "name": "Anitha",
  "email": "anitha@gmail.com",
  "mobile": "9999999999"
}
```

Step 4: Create Remote Database using MongoDB Atlas

1. Go to MongoDB Atlas website
2. Create free account
3. Create new cluster
4. Click **Connect → MongoDB Compass**

Connection string example:

mongodb+srv://username:password@cluster0.mongodb.net/movieBookingDB

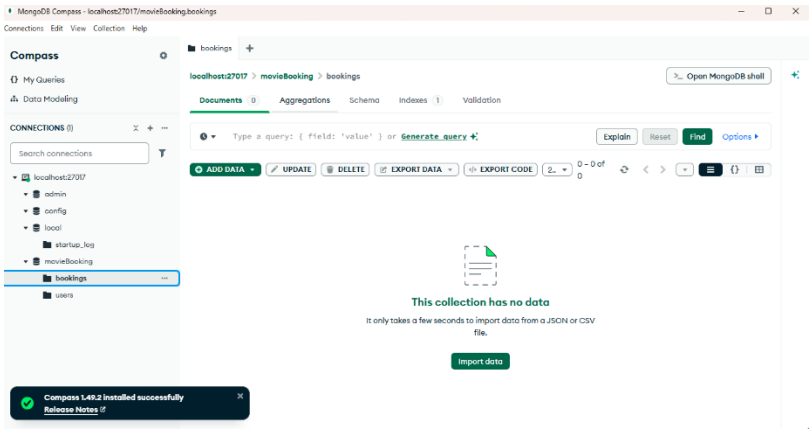
5. Paste connection string in MongoDB Compass

6. Click Connect
7. Create database and collection same as local

Step 5: Node.js Example to Connect Remote MongoDB

```
const mongoose = require("mongoose");


mongoose.connect("mongodb+srv://username:password@cluster0.mongodb.net/movieBookingDB")

.then(() => console.log("Remote MongoDB Connected"))

.catch(err => console.log(err));
```

Output:



Result:

Thus, MongoDB was successfully installed, and both local and remote databases were created using MongoDB Compass and MongoDB Atlas. Data was inserted and retrieved successfully from both databases.

Ex 12: Create a full stack Web Application – Movie Ticket

Date: 04/03/26

Aim: To design and implement a **User Management Module** using **Node.js**, **Express.js**, and **MongoDB**, enabling user registration, login, and data management with secure storage in a MongoDB database.

Source Code / Commands:

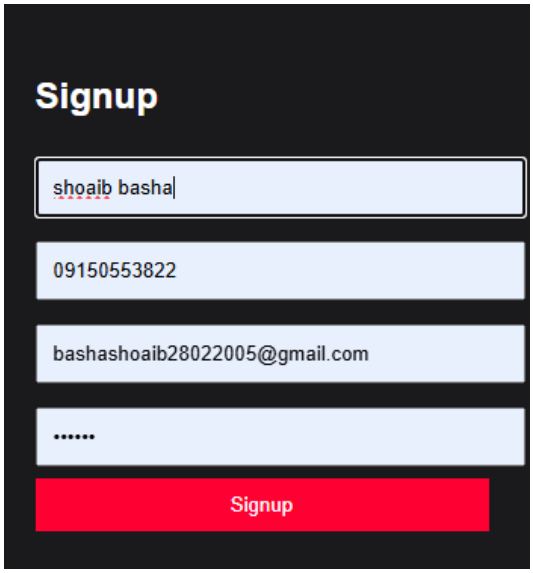
(server.js)

```
const express = require('express');

const mongoose = require('mongoose');

const bcrypt = require('bcryptjs');


const app = express();

app.use(express.json());


// Connect MongoDB

mongoose.connect('mongodb://127.0.0.1:27017/userdb')

.then(() => console.log("MongoDB Connected"))

.catch(err => console.log(err));


// User Schema

const userSchema = new mongoose.Schema({

  name: String,

  email: String,

  password: String

});


const User = mongoose.model('User', userSchema);
```

```
// Register
app.post('/register', async (req, res) => {
  const { name, email, password } = req.body;

  const existingUser = await User.findOne({ email });
  if (existingUser)
    return res.json({ message: "User already exists" });

  const hashedPassword = await bcrypt.hash(password, 10);

  const user = new User({ name, email, password: hashedPassword });
  await user.save();

  res.json({ message: "User Registered Successfully" });
});

// Login
app.post('/login', async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });
  if (!user)
    return res.json({ message: "Invalid Email" });

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch)
    return res.json({ message: "Invalid Password" });
```

```
  res.json({ message: "Login Successful" });
});

// Server
app.listen(5000, () => console.log("Server running on port 5000"));
```

Output:



Result:

The User Registration and Login system was successfully implemented using Node.js, Express.js, and MongoDB.
User data is stored in MongoDB with encrypted passwords, and login authentication works correctly.